Node.js

Handwritten Notes

1. What is Node.js ?

- Node.js is an open-source, cross-platform JavaScript runtime environment.
- It allows us to run JavaScript code outside the browser.
- It is built on Chrome's V8 JavaScript engine.
- It is used to build fast, scalable and high-performance server-side applications.

2. Why Node.js was Created ?

- Traditional servers (like Apache) create a new thread for every request.
- This leads to high memory usage and low performance.
- Node.js was created to handle multiple requests using a single thread with non-blocking I/O.
- It is lightweight and highly efficient.

3. Features of Node.js

- ✓ Asynchronous and Event-Driven
- ✓ Non-blocking I/O
- ✓ Single Threaded (with Event Loop)
- ✓ Highly Scalable
- ✓ Cross-platform
- ✓ Fast Execution (V8 Engine)
- ✓ Large Ecosystem (NPM)

4. Advantages

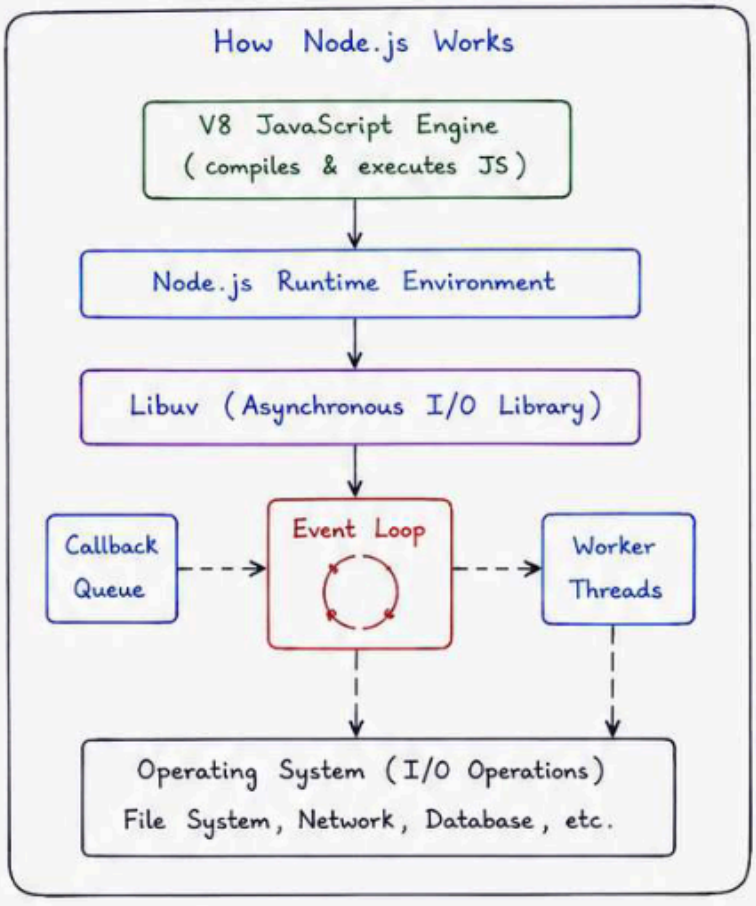
- High performance
- Scalable and efficient
- Full-stack JavaScript (same language for frontend & backend)
- Large community support
- Ideal for real-time applications

5. Where Node.js is Used ?

- Web servers
- APIs and Microservices
- Real-time applications (Chat, Gaming, Collaboration)
- Streaming applications
- CLI tools
- IoT applications

6. Node.js Architecture (How it Works)

- V8 Engine : Compiles JavaScript code into machine code.
- Node.js Runtime : Provides the environment to execute JavaScript code.
- Libuv : C library that provides non-blocking I/O operations.
- Event Loop : Handles asynchronous operations and callbacks.
- OS (Operating System) : Performs actual I/O operations (file, network, database etc.).



Node.js

Handwritten Notes

2. Installation & First Program

2.1 Install Node.js

- Go to https://nodejs.org
- Download LTS version for your OS.
- Run the installer and follow the steps.
- After installation, Node.js and npm will be available in your terminal / command prompt.

2.2 Verify Installation

Open terminal and run:

```
node -v   → shows Node.js version
npm -v    → shows npm version
```

2.3 REPL (Read Eval Print Loop)

- Node.js comes with a built-in terminal called REPL.
- It allows us to execute JavaScript code interactively.
- To start REPL, type:

```
node
```

- To exit REPL, press:

```
.exit   or   Ctrl + D
```

2.4 First Node.js Program

1. Create a file: app.js
2. Write the following code:

```
// app.js
console.log("Hello, Node.js!");
console.log("This is my first program.");
```

3. Run the program:

```
node app.js
```

4. Output:

```
Hello, Node.js!
This is my first program.
```

2.5 Running JavaScript Files

- Save your code in a .js file.
- Open terminal in the same folder.
- Run the file using:

```
node filename.js
```

Example:

```
node app.js
```

2.6 Console Methods

- `console.log()` → prints normal message
- `console.error()` → prints error message
- `console.warn()` → prints warning message
- `console.info()` → prints info message
- `console.table()` → prints data in table format
- `console.clear()` → clears the console

2.7 Global Objects

- global
  - In Node.js, 'global' is the global object.
  - Variables declared with var become properties of global object.
- __dirname
  - Gives the current directory path of the file.
- __filename
  - Gives the current file name with full path.

Node.js

Handwritten Notes

3. Modules in Node.js

- Modules are blocks of code that perform a specific task.
  Node.js has a modular architecture.

3.1 Types of Modules

1. Built-in Modules — Provided by Node.js
2. Custom Modules — Created by developers

3.2 Built-in Modules (Examples)

- fs — File System
- path — Working with file paths
- http — Create HTTP server
- os — Operating system related info
- url — URL handling
- events — Event handling
- util — Utility functions

3.3 Creating a Custom Module

1. Create a file → math.js

```
// math.js
const add = (a, b) => a + b;
const sub = (a, b) => a - b;

module.exports = {
   add,
   sub
};
```

2. Use the module in another file → app.js

```
// app.js
const math = require("./math");

console.log(math.add(5, 3));    // 8
console.log(math.sub(5, 3));    // 2
```

3.4 How require() Works

- When we use require(), Node.js looks for the module in the following order:
  1. Core modules (built-in)
  2. Local files or folders
  3. node_modules folder

3.5 module.exports vs exports

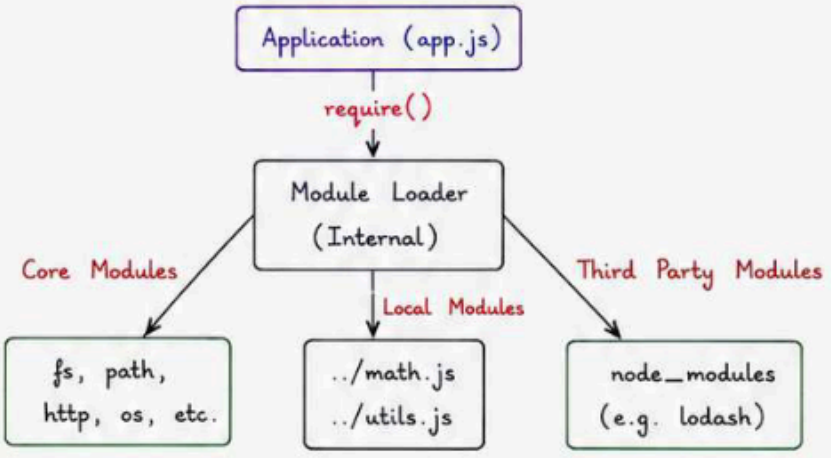
- module.exports is the recommended way to export.
- exports is a shortcut.

```
// Using module.exports
module.exports = {
   name: "Aman",
   age: 25
};
```

```
// Using exports
exports.name = "Aman";
exports.age = 25;
```

Note:
If we assign directly → exports = {}, it will not work.

3.6 Module System in Node.js



3.7 Benefits of Modules

- Code reusability
- Better organization
- Easy maintenance
- Improves readability

Node.js

Handwritten Notes

4. File System (fs Module)

The fs (File System) module allows us to work with files and directories on the file system.

4.1 Importing fs Module

```
const fs = require('fs');              // for async
const fs = require('fs').promises;     // for promise
```

4.2 Reading Files

- Asynchronous

```
fs.readFile('example.txt', 'utf8', (err, data) => {
    if (err) console.error(err);
    else console.log(data);
});
```

- Synchronous

```
try {
    const data = fs.readFileSync('example.txt', 'utf8');
    console.log(data);
} catch (err) {
    console.error(err);
}
```

4.3 Writing Files

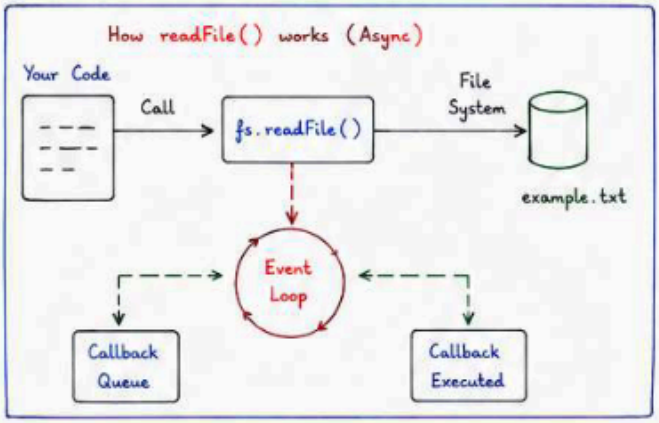
- Asynchronous

```
fs.writeFile('example.txt', 'Hello Node.js', (err) => {
    if (err) console.error(err);
    else console.log("File written successfully");
});
```

- Synchronous

```
fs.writeFileSync('example.txt', 'Hello Node.js');
console.log("File written successfully");
```

> **Note**
> Always use asynchronous methods in real applications to avoid blocking the event loop.



4.4 Appending to Files

- Asynchronous

```
fs.appendFile('example.txt', "\nNew line added', (err) => {
    if (err) console.error(err);
    else console.log("Data appended");
});
```

- Synchronous

```
fs.appendFileSync('example.txt', "\nNew line added');
console.log("Data appended");
```

4.5 Other File Operations

- Delete File : `fs.unlink('file.txt', (err) => { ... });` / `fs.unlinkSync('file.txt');`
- Rename File : `fs.rename('old.txt','new.txt', (err) => { ... });` / `fs.renameSync('old.txt','new.txt');`
- Make Directory : `fs.mkdir('myFolder', (err) => { ... });` / `fs.mkdirSync('myFolder');`
- Read Directory : `fs.readdir('myFolder', (err, files) => { ... });` / `fs.readdirSync('myFolder');`

4.6 fs Methods : Sync vs Async

Operation	Asynchronous (Non-blocking)	Synchronous (Blocking)
Read File	fs.readFile()	fs.readFileSync()
Write File	fs.writeFile()	fs.writeFileSync()
Append File	fs.appendFile()	fs.appendFileSync()
Delete File	fs.unlink()	fs.unlinkSync()
Rename File	fs.rename()	fs.renameSync()
Make Directory	fs.mkdir()	fs.mkdirSync()
Read Directory	fs.readdir()	fs.readdirSync()

4.7 Path Module (Working with Paths)

The path module helps us to work with file paths.

```
const path = require('path');
```

- `path.join('a','b','c')` → joins path segments
- `path.resolve('a','b')` → resolves to absolute path
- `path.basename('/a/b/c.txt')` → returns `'c.txt'`
- `path.dirname('/a/b/c.txt')` → returns `'/a/b'`
- `path.extname('c.txt')` → returns `'.txt'`

Node.js

Handwritten Notes

5. NPM & Package Management

NPM (Node Package Manager) is the default package manager for Node.js. It helps us to install, share and manage packages.

5.1 What is npm ?

- npm is the largest ecosystem of open-source packages.
- It comes with Node.js installation.
- It is used to install and manage dependencies in our project.

5.2 package.json

- Every Node.js project has a package.json file.
- It contains metadata about the project and its dependencies.
- We can create it manually or using :

```
npm init
```

Example (package.json)

```
{
  "name": "node-demo",
  "version": "1.0.0",
  "description": "My first Node.js project",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
  "author": "Aman",
  "license": "ISC",
  "dependencies": {},
  "devDependencies": {}
}
```

5.3 Installing Packages

- To install a package :

```
npm install <package-name>
```

- Example :

```
npm install lodash
```

- This will install the package inside node_modules folder and add it to package.json.

5.4 Dev Dependencies

- Some packages are needed only for development (not in production).
- Use --save-dev (or -D) to install dev dependencies.

```
npm install <package-name> --save-dev
```

Example :

```
npm install nodemon --save-dev
```

5.5 Other Useful Commands

Command	Description
npm install	Install all dependencies from package.json
npm install <pkg>	Install a package
npm uninstall <pkg>	Remove a package
npm update <pkg>	Update a package
npm outdated	Check outdated packages
npm list	List all installed packages
npm search <pkg>	Search package in npm registry

5.6 node_modules Folder

- When we install any package, it is stored in the node_modules folder.
- This folder can be very large.
- We should not upload it to GitHub.

5.7 package-lock.json

- Automatically generated when we install packages.
- It locks the exact version of dependencies.
- Ensures same version for everyone working on the project.

5.8 Project Structure

```
node-demo/
├── node_modules/     (installed packages)
├── package.json      (project metadata)
├── package-lock.json (lock file)
└── app.js            (main file)
```

Basic Structure

Note :
Always commit package.json and package-lock.json, but never commit node_modules folder.

Node.js

Handwritten Notes

6. Asynchronous Programming in Node.js

Node.js is built on asynchronous, non-blocking I/O model. It allows it to handle multiple operations at the same time.

6.1 Callback Function

- In Node.js, many functions accept a callback function as the last argument.
- The callback is executed after the operation is completed.

Example :

```
const fs = require('fs');

fs.readFile('example.txt', 'utf8', (err, data) => {
   if (err) {
      console.error(err);
   } else {
      console.log(data);
   }
});
```

6.2 Callback Hell

- When we use multiple nested callbacks, the code becomes difficult to read and maintain.
- This is called Callback Hell.

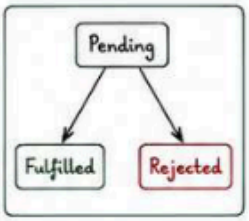
Example :

```
fs.readFile('file1.txt', 'utf8', (err, data1) => {
  if (!err) {
    fs.readFile('file2.txt', 'utf8', (err, data2) => {
      if (!err) {
        fs.readFile('file3.txt', 'utf8', (err, data3) => {
          if (!err) {
            console.log(data3);
          }
        });
      }
    });
  }
});
```

6.3 Promise

- A Promise is an object that represents the eventual completion (or failure) of an asynchronous operation.
- It has 3 states :

1. Pending - Initial state
2. Fulfilled - Operation completed successfully
3. Rejected - Operation failed



6.4 Creating a Promise

```
const myPromise = new Promise((resolve, reject) => {
   let success = true;

   if (success) {
      resolve('Operation Successful');
   } else {
      reject('Operation Failed');
   }
});

myPromise
   .then((result) => console.log(result))
   .catch((error) => console.error(error));
```

6.5 async / await

- async/await is a more modern and clean way to work with Promises.
- It makes asynchronous code look synchronous.

Example :

```
const fs = require('fs').promises;  // fs with Promise API

async function readFileData() {
  try {
     const data = await fs.readFile('example.txt', 'utf8');
     console.log(data);
  } catch (err) {
     console.error(err);
  }
}
readFileData();
```

6.6 Comparison

	Callback	Promise	async/await
Readability	Low (nested)	Medium	High
Error Handling	Hard	Better (using .catch)	Easy (using try...catch)
Code Style	Nested Callbacks	Chained .then()	Cleaner (Synchronous Style)
Best Use	Small tasks	When chaining multiple tasks	Modern way for complex logic

6.7 try...catch with async/await

```
async function getData() {
  try {
     const data = await fs.readFile('notfound.txt', 'utf8');
     console.log(data);
  } catch (err) {
     console.error('Error:', err.message);
  }
}
getData();
```

Recommended way to handle errors in async code.

Node.js

Handwritten Notes

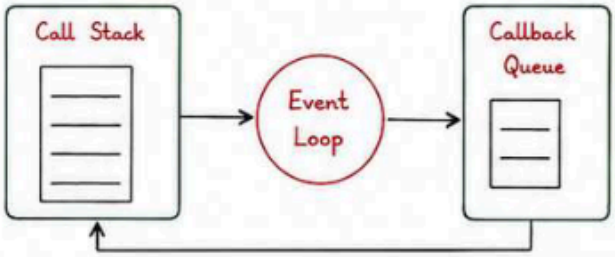
7. Event Loop & EventEmitter

7.1 Node.js is Single Threaded

- Node.js uses a single thread to handle requests.
- It uses an event loop to handle multiple operations without blocking.

7.2 Event Loop in Node.js

- The event loop is the heart of Node.js.
- It allows Node.js to perform non-blocking I/O operations.



How it works:

1. Code is pushed into Call Stack.
2. When an async task is encountered, it is offloaded to the system (APIs).
3. Once the task is complete, its callback is pushed into Callback Queue.
4. Event Loop checks if Call Stack is empty.
5. If yes, it pushes the callback from Queue to Call Stack for execution.

7.3 Phases of Event Loop

1. timers	→ Executes setTimeout() and setInterval() callbacks.
2. pending callbacks	→ Executes I/O callbacks deferred to next loop iteration.
3. idle, prepare	→ Internal use only.
4. poll	→ Retrieves new I/O events; executes I/O callbacks.
5. check	→ setImmediate() callbacks.
6. close callbacks	→ Executes close event callbacks (e.g., socket.on('close'))

Note:
The event loop continues to run as long as there are pending tasks or handles.

7.4 EventEmitter Module

- Node.js provides events module to work with event-driven architecture.
- It allows us to create, listen and emit events.

```
const EventEmitter = require('events');
const emitter = new EventEmitter();
```

7.5 Common Methods

`emitter.on(event, listener)` → Registers a listener for an event.

`emitter.emit(event, data)` → Triggers the event and sends data.

`emitter.once(event, listener)` → Listens only once.

`emitter.removeListener(event, listener)` → Removes a specific listener.

`emitter.removeAllListeners(event)` → Removes all listeners for an event.

7.6 Example

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

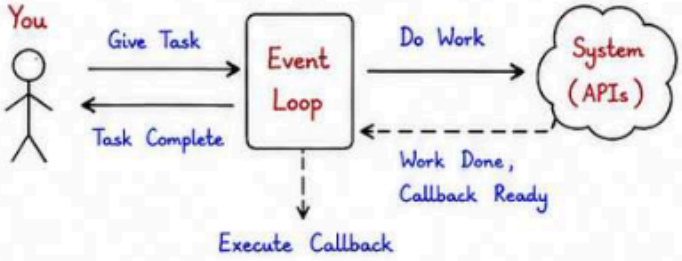
// Listen to event
emitter.on('greet', function(name) {
   console.log('Hello ' + name);
});

// Emit event
emitter.emit('greet', 'Aman');    // Output: Hello Aman

// Listen once
emitter.once('welcome', () => {
   console.log('Welcome Event Triggered');
});

emitter.emit('welcome');    // Output: Welcome Event Triggered
emitter.emit('welcome');    // Not printed second time
```

7.7 Real Life Analogy



Node.js Handwritten Notes

8. HTTP Module & Creating Server

Node.js has a built-in http module which allows us to create HTTP servers and handle requests and responses.

8.1 HTTP Module

- http is a core module in Node.js.
- It enables us to create web servers without using any external library.
- It is low-level and uses streams internally.

8.2 Creating a Basic Server

```
const http = require('http');
const server = http.createServer((req, res) => {
  // handle request and send response
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');
  res.end('Hello from Node.js Server!');
});

server.listen(3000, () => {
  console.log('Server is running on port 3000');
});
```

Open browser and visit: http://localhost:3000
You will see the response.

8.3 Request Object (req)

- Contains information about the incoming request.
- Common properties:

Property	Description
req.method	HTTP method (GET, POST, etc.)
req.url	URL of the request
req.headers	Headers sent by the client
req.query	Query string (if any)
req.body	Data sent in request body (for POST, etc.)

8.4 HTTP Methods

Method	Purpose
GET	Request data from the server
POST	Send data to the server
PUT	Update data on the server
DELETE	Delete resource from the server
PATCH	Partially update data

8.5 Response Object (res)

- Used to send data back to the client.
- Common methods:

Method	Description
res.statusCode	Set HTTP status code
res.setHeader()	Set response headers
res.write()	Write data to response
res.end()	End the response
res.json()	Send JSON response

8.6 Sending Different Types of Response

- Plain Text

```
res.statusCode = 200;
res.setHeader('Content-Type', 'text/plain');
res.end('Hello World');
```

- HTML

```
res.statusCode = 200;
res.setHeader('Content-Type', 'text/html');
res.end('<h1>Hello from Node.js</h1>');
```

- JSON

```
res.statusCode = 200;
res.setHeader('Content-Type', 'application/json');
res.end(JSON.stringify({ name: 'Aman', age: 25 }));
```

8.7 Basic Routing

We can handle different routes based on `req.url` and `req.method`.

```
const http = require('http');
const server = http.createServer((req, res) => {
  if (req.url === '/' && req.method === 'GET') {
    res.end('Home Page');
  } else if (req.url === '/about') {
    res.end('About Page');
  } else if (req.url === '/contact') {
    res.end('Contact Page');
  } else {
    res.statusCode = 404;
    res.end('Page Not Found');
  }
});

server.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

8.8 Common HTTP Status Codes

- 200 OK
- 201 Created
- 204 No Content
- 301 Moved Permanently
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 500 Internal Server Error
- 502 Bad Gateway
- 503 Service Unavailable

Node.js Handwritten Notes

9. Modules in Node.js

Modules help us to split the code into multiple files and reuse them.

9.1 Core Modules (Built-in)

Node.js comes with many built-in modules.
Some commonly used ones:

Module	Purpose
fs	Work with files
path	Work with file and directory paths
http	Create HTTP servers and clients
os	Get operating system information
util	Utility functions
events	Work with events
url	Parse and format URLs

9.2 Using Core Module (Example - fs)

```
const fs = require('fs');
// Read a file
fs.readFile('example.txt', 'utf8', (err, data) => {
    if (err) {
        console.error(err);
    } else {
        console.log(data);
    }
});
```

9.3 Creating Your Own Module

- We can create our own modules.
- Use module.exports to export functions or objects.

Example: math.js

```
// math.js
function add(a, b) {
    return a + b;
}
function multiply(a, b) {
    return a * b;
}
module.exports = { add, multiply };
```

Using the module in another file:

```
// app.js
const math = require('./math');
console.log(math.add(5, 3));      // 8
console.log(math.multiply(5, 3)); // 15
```

9.4 module.exports vs exports

module.exports	exports
• You can assign any value (object, function, etc.).	• Shortcuts to module.exports.
• Overwrites the entire export object.	• Only adds properties to existing export object.
	• Cannot overwrite.

9.5 require()

- Used to import modules.
- Works synchronously.
- Returns the exported value from the module.

Example:

```
const fs = require('fs');              // core module
const myModule = require('./math');    // custom module
```

9.6 Built-in Module Example - path

```
const path = require('path');
console.log(path.join('a', 'b', 'c.txt'));   // a/b/c.txt
console.log(path.basename('/a/b/c.txt'));    // c.txt
console.log(path.extname('c.txt'));          // .txt
```

9.7 Global Object

- Node.js provides a global object.
- Some useful globals:

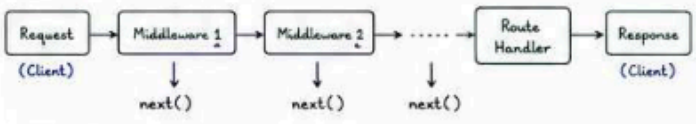
Global	Purpose
__filename	Current file name
__dirname	Current directory name
process	Information about current process
console	Used for logging
setTimeout()	Execute code after a delay
setInterval()	Repeat code after a delay

10. Middleware in Node.js (with Express.js)

10.1 What is Middleware ?

- Middleware functions are functions that have access to the request and response objects.
- They can execute any code, make changes to the request and response, end the request-response cycle, or call the next middleware in the stack.

10.2 Middleware Flow



10.3 Types of Middleware

1. Application-level middleware
2. Router-level middleware
3. Error-handling middleware
4. Built-in middleware
5. Third-party middleware

10.4 Simple Middleware Example

```
const express = require('express');
const app = express();
// Middleware
app.use((req, res, next) => {
    console.log('Request received at ' + new Date().toISOString());
    next();    // pass control to next middleware
});
// Route
app.get('/', (req, res) => {
    res.send('Hello Middleware!');
});

app.listen(3000, () => console.log('Server running on port 3000'));
```

10.5 Error Handling Middleware

```
app.use((err, req, res, next) => {
    console.error(err.stack);
    res.status(500).send('Something went wrong!');
});
```

Always keep this at the end of your middleware stack.

Node.js

Handwritten Notes

11. Streams in Node.js

Streams are used to read or write data piece by piece instead of all at once. Useful for large files.

11.1 Types of Streams

- Readable - Read data from a source
- Writable - Write data to a destination
- Duplex - Both readable and writable
- Transform - Modify or transform the data

11.2 Example - Read a File using Stream

```
const fs = require('fs');
const readStream = fs.createReadStream('large.txt', 'utf8');
readStream.on('data', (chunk) => {
   console.log('Received chunk of size:', chunk.length);
});
readStream.on('end', () => {
   console.log('Reading completed');
});
readStream.on('error', (err) => {
   console.error('Error:', err.message);
});
```

11.3 Example - Write to a File using Stream

```
const fs = require('fs');
const writeStream = fs.createWriteStream('output.txt');
writeStream.write('Hello Node.js Streams!\n');
writeStream.write('This is line 2\n');
writeStream.end('This is the last line.');
writeStream.on('finish', () => {
   console.log('Writing completed');
});
writeStream.on('error', (err) => {
   console.error('Error:', err.message);
});
```

12. Buffer in Node.js

Buffer is used to handle binary data.

12.1 Create a Buffer

```
const buf = Buffer.from('Hello');  // create buffer
```

12.2 Buffer Methods

Method	Description
Buffer.from()	Create buffer from string or array
buf.toString()	Convert buffer to string
buf.length	Get size of buffer
buf.slice(a,b)	Slice buffer from a to b
buf.copy(target)	Copy buffer to another buffer
buf.write(str)	Write string to buffer

Quick Note

- Node.js is not a language, it is a runtime environment.
- Everything in Node.js is non-blocking and event-driven.
- Use modules to keep code organized and reusable.
- Always handle errors using try...catch or .catch().
- Use streams and buffers for better performance.

13. Error Handling

13.1 Try...Catch (Synchronous)

```
try {
   const x = 10 / 0;
   if (!isFinite(x)) throw new Error('Something went wrong');
} catch (err) {
   console.error('Error:', err.message);
}
```

13.2 Handling Errors in Callbacks

```
fs.readFile('file.txt', 'utf8', (err, data) => {
   if (err) {
      console.error('Error reading file:', err.message);
      return;
   }
   console.log(data);
});
```

13.3 Handling Errors in Promises

```
fs.promises.readFile('file.txt', 'utf8')
  .then(data => console.log(data))
  .catch(err => console.error('Error:', err.message));
```

14. Common Console Methods

Method	Use
console.log()	Normal output
console.error()	Error messages
console.warn()	Warning messages
console.info()	Informational messages
console.table()	Print data in table format

15. Useful Shortcuts

- ctrl + c → Stop the server in terminal
- node app.js → Run the application
- nodemon app.js → Auto restart on file changes
- npm init -y → Create package.json quickly
- npm i <pkg> → Install a package
- npm list → List installed packages

16. Summary

- ☑ Node.js is built on Chrome V8 engine.
- ☑ It is event-driven, non-blocking and asynchronous.
- ☑ Core modules help us do many tasks without installing extra packages.
- ☑ npm is used to manage packages.
- ☑ HTTP module helps to create web servers.
- ☑ Streams and Buffers help in handling large data.
- ☑ Always handle errors properly.
- ☑ Practice by building small projects.

Remember: Practice is the key to mastering Node.js. Build small projects and keep learning! ☺